The Agentic Loop Playbook

Building Self-Prompting, Self-Correcting Claude Workflows

A step-by-step guide to designing loops where Claude plans, builds, verifies, and learns from every scenario — drawn from Anthropic engineering practice.

Version 1.0 | June 2026

What this playbook covers

- The anatomy of the agentic loop and the principles Anthropic uses internally.
- Step-by-step environment setup (CLAUDE.md, hooks, skills, subagents, permissions).
- The verification rule that separates loops you babysit from loops that run alone.
- Self-prompting patterns: orchestrator–workers, evaluator–optimizer, fan-out, and more.
- How to make Claude learn from every scenario and compound knowledge across sessions.
- A prompt template library, anti-pattern guide, and quick-reference checklist.

1. What an Agentic Loop Actually Is

An agentic loop is the cycle every Claude agent runs: **gather context** → **take action** → **verify the work** → **repeat**. Unlike a chatbot that answers and waits, an agent reads files, runs commands, makes changes, checks the result, and decides for itself what to do next. The quality of a loop is determined not by clever prompting tricks but by how well each of those four phases is engineered.

Loop phase	What happens	Your engineering lever
Gather context	Claude reads files, docs, memory, and prior results	CLAUDE.md, skills, subagents, scoped prompts
Take action	Claude edits files, runs commands, calls tools	Tool design, permissions, sandboxing
Verify work	Claude runs a check that returns pass / fail	Tests, linters, builds, screenshots, evaluator agents
Repeat	Claude reads the result and iterates until the check passes	/goal conditions, Stop hooks, loop scripts

Three core principles

- **Simplicity.** The most successful implementations use simple, composable patterns — not complex frameworks. Start with a single loop; add agents only when one demonstrably isn't enough.
- **Transparency.** Show the agent's planning steps explicitly. If you can't see the plan, you can't correct it before it becomes wrong code.
- **A well-crafted agent–computer interface.** Document and test your tools as carefully as you would a human-facing API. Most agent failures are really tool-documentation failures.

> **The one constraint behind every best practice**
>
> Claude's context window fills fast, and performance degrades as it fills. Nearly every technique in this playbook — subagents, /clear, scoped prompts, compaction, fresh-context reviewers — exists to protect the context window. Treat context as infrastructure.

2. Step-by-Step: Build the Foundation First

A loop is only as good as the environment it runs in. Claude pulls context automatically from your project; if that environment is carefully designed, performance compounds. Do these steps once per project, in order, before attempting any autonomous loop.

1. **Create CLAUDE.md** in the project root (run /init to generate a starter). Include build/test commands Claude can't guess, code-style rules that differ from defaults, repository etiquette, architectural decisions, and known gotchas. Keep it short — for every line ask: “would removing this cause Claude to make mistakes?” Bloated files cause Claude to ignore your actual instructions.

2. **Configure permissions.** Allowlist safe commands (npm run lint, git commit), use auto mode for classifier-reviewed autonomy, or enable sandboxing for OS-level isolation. A loop that stops every 30 seconds for approval is not a loop.

3. **Install CLI tools** (gh, aws, etc.). CLIs are the most context-efficient way for Claude to interact with external services, and Claude can teach itself unfamiliar ones via --help.

4. **Add hooks** for anything that must happen every time with zero exceptions — e.g. run the linter after every edit, block writes to protected folders. CLAUDE.md instructions are advisory; hooks are deterministic. Ask Claude to write the hooks for you.

5. **Create skills** (.claude/skills/*/SKILL.md) for domain knowledge and repeatable workflows that only matter sometimes. Claude loads them on demand without bloating every session.

6. **Define subagents** (.claude/agents/*.md) for specialized roles — researcher, security reviewer, test writer. Each runs in its own context with its own allowed tools.

> **Why this order matters**
>
> CLAUDE.md shapes every turn; permissions decide whether the loop can run unattended; hooks make quality deterministic; skills and subagents keep the main context clean. Each layer multiplies the value of the next.

3. The Golden Rule: Give Claude a Check It Can Run

This is the single highest-leverage practice in agentic engineering. Claude stops when the work *looks* done. Without a check it can run, “looks done” is the only signal available — and you become the verification loop, catching every mistake by hand. Give Claude something that produces a pass or fail, and the loop closes on its own: Claude does the work, runs the check, reads the result, and iterates until it passes.

A check is anything that returns a readable signal: a test suite, a build exit code, a linter, a script that diffs output against a fixture, or a browser screenshot compared against a design. The best feedback is rules-based — clearly defined rules for the output, plus which rules failed and why.

Four escalating ways to gate the loop on the check

Level	Mechanism	When to use
1. In the prompt	“Run the tests after implementing and fix failures.”	Any task, today, zero setup
2. Goal condition (/goal)	A separate evaluator re-checks the condition after every turn; Claude keeps working until it holds	Sessions that must not stop at soft completion
3. Stop hook	Your script runs as a deterministic gate and blocks the turn from ending until it passes	Unattended runs needing hard guarantees
4. Adversarial subagent	A fresh-context agent reviews the diff against the plan and reports gaps — the worker never grades its own work	Long autonomous runs; final sign-off

> **Demand evidence, not assertions**
>
> Have Claude show the test output, the command it ran and what it returned, or a screenshot — not just “done.” Reviewing evidence is faster than re-verifying yourself, and it works for sessions you never watched. Anthropic's own security-research agents run every finding through a self-correction loop in which Claude attempts to disprove its own report — filtering false positives before a human ever sees them.

4. The Core Loop: Explore → Plan → Implement → Verify → Commit

Internal workflows converge on the same architecture. Letting Claude jump straight to code produces solutions to the wrong problem; separating research from implementation is what keeps a loop on target.

1. **Explore (plan mode).** Have Claude read the relevant files and answer questions without making changes: “Read /src/auth and understand how we handle sessions. Don't write code yet.”
2. **Plan.** Ask for a detailed implementation plan naming the files that change, the order, and the verification step. Edit the plan directly before approving — minutes here save hours later.
3. **Implement.** Exit plan mode and let Claude code against its plan, running the check as it goes: “Implement the OAuth flow from your plan. Write tests for the callback handler, run the suite, fix failures.”
4. **Verify independently.** Spawn a fresh-context reviewer: “Use a subagent to review the diff against PLAN.md. Check every requirement is implemented, edge cases have tests, and nothing out of scope changed. Report gaps, not style preferences.”
5. **Commit and capture.** Commit with a descriptive message, open a PR — and record anything learned in CLAUDE.md (see Section 6) so the next loop starts smarter.

Before you build: let Claude interview you

For larger features, invert the prompt. Have Claude interview you with structured questions about implementation, UX, edge cases, and tradeoffs until everything is covered, then write a complete spec to SPEC.md. Then start a fresh session to execute the spec — clean context focused entirely on implementation, with a written contract to verify against. The most useful specs are self-contained: they name files and interfaces, state what is out of scope, and end with an end-to-end verification step.

PROMPT TEMPLATE — Interview-to-spec

```
I want to build [brief description]. Interview me in detail using the

AskUserQuestion tool. Ask about technical implementation, UI/UX, edge cases,

concerns, and tradeoffs. Don't ask obvious questions — dig into the hard parts

I might not have considered. Keep interviewing until we've covered everything,

then write a complete spec to SPEC.md.
```

5. Self-Prompting Patterns: How Claude Drives Itself

Anthropic's “Building Effective Agents” defines the composable patterns from which every serious agentic system is assembled. Master these five, then combine them.

Pattern	How it works	Best for
Prompt chaining	Each step's output becomes the next step's input, with optional gates between steps	Tasks that decompose into fixed, sequential subtasks
Routing	A classifier directs each input to a specialized prompt/handler	Heterogeneous inputs (triage, support, mixed work items)
Parallelization	Independent subtasks run simultaneously; results are aggregated or voted on	Speed, or multiple perspectives on the same question

Pattern	How it works	Best for
Orchestrator–workers	A lead agent dynamically decomposes the task and delegates to workers it spawns	Unpredictable scope — multi-file changes, research
Evaluator–optimizer	One agent generates, a second evaluates against criteria; repeat until quality holds	Iterative refinement with clear evaluation criteria

The flagship example: a multi-agent research system

Anthropic's Research feature is an orchestrator–worker loop: a lead agent (Opus) analyzes the query, develops a strategy, spawns 3–5 specialized subagents (Sonnet) that explore in parallel, then synthesizes their findings with a separate citation pass. This outperformed a single-agent Opus baseline by 90.2% on internal research evals. The published lessons apply directly to any loop you build:

- **Think like your agents.** Simulate a step yourself with only the context the agent will have. If you can't do the step with that context, neither can Claude.
- **Scale effort to complexity.** Early versions spawned 50 subagents for simple queries. Tell the orchestrator explicitly how many workers and tool calls different query types deserve.
- **Teach the orchestrator to delegate.** Each subagent needs an exact role, focused instructions, only the relevant input, and a required output schema (with a concise summary). Vague delegation produces duplicated and missing work.

PROMPT TEMPLATE — Orchestrator starter

```
You are an expert Orchestrator Agent. User task: [TASK].

1. Decompose into the smallest possible independent sub-tasks.

2. For each sub-task define: exact specialized role; focused instructions;

relevant input data only; required JSON output schema with a concise

summary (<= 400 tokens).

3. Output a clear execution plan first. I will run sub-agents in parallel

and return their summaries.

4. Then synthesize into one cohesive final deliverable.

Prioritize structured outputs and minimal context per agent.
```

Match the pattern to the failure mode

Observed failure mode	Pattern that fixes it
Agentic laziness — partial progress declared “done”	Loop-until-done with a /goal condition or Stop hook
Self-preferential bias — Claude grading its own work	Adversarial verification by a fresh-context agent
Goal drift — losing the objective across many turns	Fan-out and synthesize: short-lived workers, durable orchestrator

Observed failure mode	Pattern that fixes it
Taste-based output that's hard to score absolutely	Tournament — pairwise comparison instead of absolute scores
Unknown amount of work remaining	Loop-until-done over a generated task list
Heterogeneous work items	Classify-and-act, then dedupe, then fix or escalate

6. Learning from Every Scenario: Compounding Knowledge

A loop that doesn't persist what it learns repeats its mistakes forever. Claude's context dies with the session — so learning must be written to files the next session reads. This is the closest thing to “Claude teaching itself” that exists today, and it is entirely under your control.

1. **Treat CLAUDE.md as the long-term memory.** Whenever a session reveals something repeatable — a gotcha, a convention, a command — have Claude append it: “Before finishing, update CLAUDE.md with any lesson from this session that would have saved you time if you'd known it at the start.” The file is version-controlled and compounds in value across the whole team.

2. **Prune as aggressively as you add.** If Claude already behaves correctly without a rule, delete the rule. If Claude keeps violating a rule, the file is probably too long and the rule is lost in noise. Treat CLAUDE.md like code: review it when things go wrong, and test edits by watching whether behavior shifts.

3. **Use the two-correction rule.** If you've corrected Claude twice on the same issue in one session, the context is polluted with failed approaches. /clear, write a better prompt that incorporates what you learned, and bank the lesson in CLAUDE.md. A clean session with a better prompt beats a long session with accumulated corrections almost every time.

4. **Persist working state to files, not context.** For long loops, have Claude maintain a PROGRESS.md or task-list file it re-reads each iteration: what's done, what failed and why, what's next. This survives compaction, /clear, and even moving the work to a fresh session.

5. **Promote proven workflows into skills.** When a loop shape works twice, capture it as a SKILL.md (or saved workflow) so it becomes a one-line invocation instead of a re-prompting exercise. Never re-prompt the same shape every time — that's the most common waste in agentic work.

PROMPT TEMPLATE — End-of-session learning pass

```
Before we finish: (1) Append to CLAUDE.md any repeatable lesson from this

session — commands, gotchas, conventions — phrased as a short imperative

rule. (2) Delete any CLAUDE.md rule this session proved unnecessary.

(3) Update PROGRESS.md: completed items, failed approaches and why they

failed, and the exact next step for a fresh session to pick up.
```

7. Scaling Up: Loops That Run Without You

Once one supervised loop works, multiply it. Each technique below trades setup effort for the ability to walk away.

Headless invocation — the loop primitive

`claude -p "prompt"` runs Claude non-interactively with parseable output (text, JSON, streaming JSON). This is how Claude becomes a component in CI pipelines, pre-commit hooks, cron jobs, and shell loops — including loops that generate the next prompt from the previous result.

Fan-out across many items

1. Have Claude generate the task list ("list all 2,000 files that need migrating").
2. Loop a scoped headless call over the list:

```
for file in $(cat files.txt); do

claude -p "Migrate $file from React to Vue. Run the tests. Return OK or FAIL." \

--allowedTools "Edit,Bash(git commit *)"

done
```

3. Test on 2–3 files, refine the prompt based on what goes wrong, then run at scale. --allowedTools scopes what an unattended agent can touch.

Writer / Reviewer — parallel sessions that check each other

Run two sessions (worktrees, the desktop app, or agent teams). Session A implements; Session B — with fresh, unbiased context — reviews the diff for edge cases, race conditions, and consistency with existing patterns; Session A addresses the feedback. The same split works for tests: one Claude writes the tests, another writes code to pass them. A fresh context improves review because Claude won't favor code it just wrote.

Full autonomy, safely

- **Auto mode** (claude --permission-mode auto -p "…"): a classifier model reviews each command, blocking scope escalation and hostile-content-driven actions while routine work proceeds.
- **Worktrees / cloud sessions** isolate each loop in its own checkout or VM so parallel edits never collide.
- **Agent teams** add shared task lists, messaging, and a team lead for coordinated multi-session runs.

> **Safety rails for unattended loops**
>
> Always set an explicit token budget — runaway loops are a cost failure mode, not just a quality one.
> Never let untrusted input (tickets, scraped web content) reach a high-privilege actor agent.
> Don't reach for a workflow when plain Claude Code solves the task in under five minutes.
> Tell reviewers to flag only gaps affecting correctness or stated requirements — a reviewer asked to find gaps will always find some, and chasing them all produces over-engineering.

8. Context Discipline Inside the Loop

Technique	When	Effect
/clear between unrelated tasks	Every task switch	Resets the window; prevents the "kitchen sink" session

Technique	When	Effect
Subagents for investigation	Any research or codebase exploration	Reading happens in a separate context; only a summary returns
/compact with instructions	Approaching limits mid-task	Summarizes while preserving the files, decisions, and commands you name
Checkpoints (/rewind)	After a risky experiment fails	Restore code and conversation; try a different approach cheaply
Named resumable sessions	Multi-day workstreams	Each stream keeps its own persistent context, like a branch

> **The rule of thumb**
>
> The main session should hold the plan and the decisions; everything else — exploration, bulk reading, review — belongs in a subagent, a file, or a fresh session.

9. Anti-Patterns: How Loops Fail

Anti-pattern	Symptom	Fix
Kitchen-sink session	Unrelated tasks share one context; quality decays	/clear between tasks
Correcting over and over	Same mistake survives repeated correction	After two failures: /clear + a better prompt with the lesson baked in
Over-specified CLAUDE.md	Claude ignores rules that are plainly written down	Ruthlessly prune; convert must-happen rules into hooks
Trust-then-verify gap	Plausible code that misses edge cases ships	No check, no ship — always provide verification
Infinite exploration	“Investigate X” reads hundreds of files, fills context	Scope investigations narrowly or push them into subagents
Worker grades own work	Reviews always pass; bugs always survive	Separate evaluator with fresh context, every time
No token budget	Loop runs all night, bill arrives in the morning	Explicit budgets and iteration caps on every workflow
Re-prompting the same shape	You rebuild the same loop weekly from memory	Save it: skill, workflow file, or CLAUDE.md pattern

10. Quick-Reference: The Loop-Builder's Checklist

Before the loop

- CLAUDE.md exists, is short, and is current. Permissions/auto mode configured. Hooks cover the non-negotiables.
- The task has a written spec or plan with files named and scope bounded.

- A check exists that returns pass/fail, and Claude can run it.

During the loop

- Explore and plan before implementing. Approve or edit the plan.
- Claude runs the check each iteration and shows evidence, not assertions.
- Investigation goes to subagents; the main context holds plan and decisions only.
- Two corrections on the same issue → /clear and re-prompt with the lesson included.

After the loop

- A fresh-context reviewer checked the diff against the plan.
- Lessons appended to CLAUDE.md; stale rules pruned; PROGRESS.md updated.
- If the loop shape worked, it's saved as a skill or workflow — never rebuilt from memory.

> **The compounding effect**
>
> Each pass through this checklist makes the next loop cheaper: better CLAUDE.md, sharper checks, saved workflows, cleaner prompts. Teams that run this discipline for a month find Claude completing work unattended that previously required constant supervision. The loop isn't just how Claude works — it's how your whole setup learns.

Sources — Anthropic engineering & documentation

- Claude Code: Best Practices — code.claude.com/docs/en/best-practices
- Building Effective AI Agents — anthropic.com/research/building-effective-agents
- How We Built Our Multi-Agent Research System — anthropic.com/engineering/multi-agent-research-system
- Building Agents with the Claude Agent SDK — anthropic.com/engineering/building-agents-with-the-claude-agent-sdk
- Anthropic agentic-workflows skill (Dynamic Workflows & multi-agent team patterns)